Design Document

AI-Powered Telecom Network Operations Assistant. This document describes the system **as built** (updated 2026-10-11). Where the build differs from the original plan, the difference is stated.

1. Business problem, objectives and scope

Problem. NOC teams keep dealing with the same kinds of degradation: congestion, poor signal, high latency, packet loss, dropped calls, handover failures, equipment faults. Knowledge of what similar situations looked like, why they happened and what fixed them is buried in thousands of historical KPI records. Searching them by hand is slow, which raises the mean time to repair and makes triage inconsistent.

Objective. Shorten triage and standardize decisions. An engineer describes a problem in plain language and gets back:

- the most similar historical cases;
- a classification, root-cause hypotheses and the impact;
- fault-risk and priority scores;
- a prioritized, cited and validated list of actions.

The system also improves from engineer feedback.

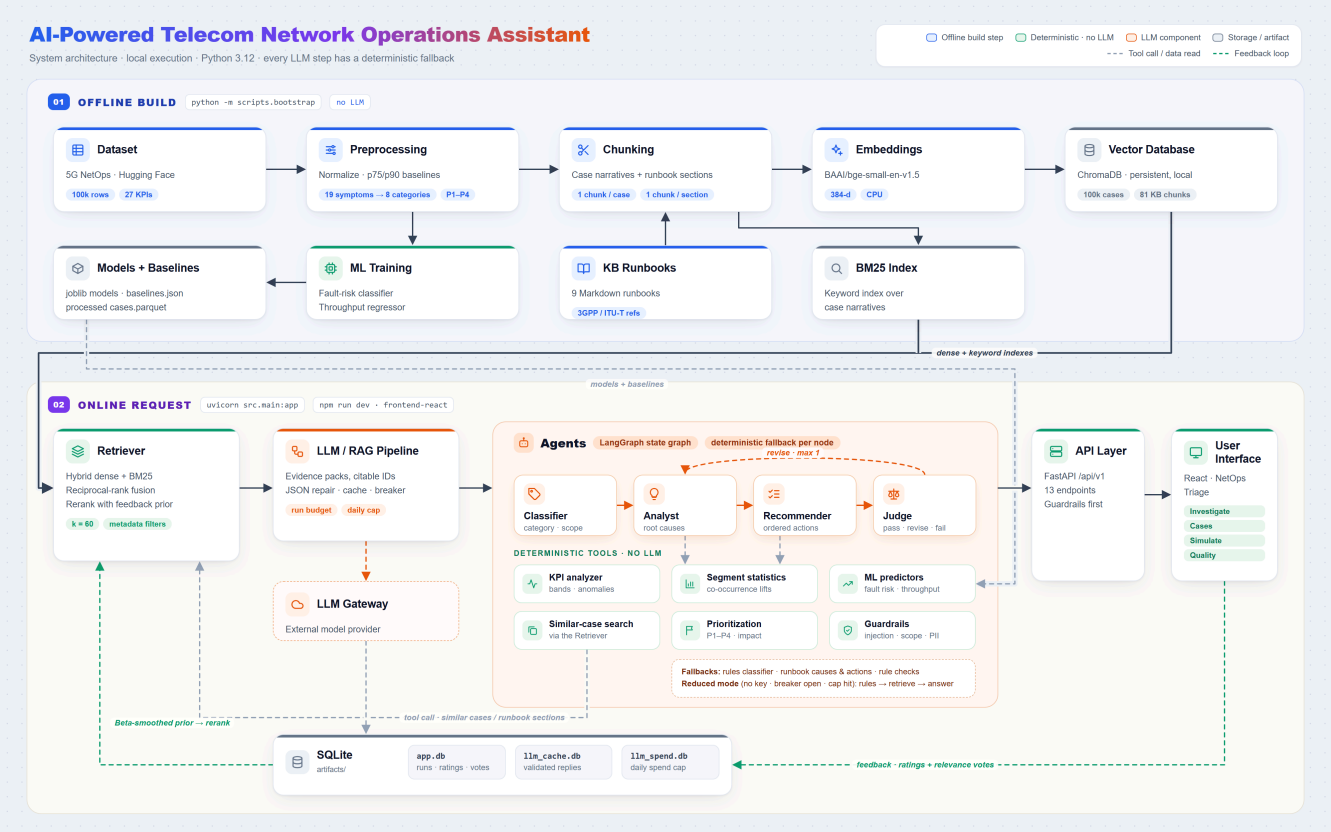
User	Needs	Main views
L1/L2 NOC engineer	Fast triage, similar cases, next actions	Investigate, Cases
RAN / transport optimization engineer	Contributing factors, KPI vs baseline, what-if predictions	Investigate, Simulate
NOC shift lead	Priority, fault risk, users and traffic affected	Investigate, Quality

Scope. Local, single-user, CPU-only; FastAPI backend, React frontend (NetOps Triage), DeepEval evaluation. Out of scope: authentication, Docker/cloud deployment, CI/CD, fine-tuning, streaming ingestion.

Assumptions.

- **A1** One dataset row is one historical cell-level KPI snapshot, called a *case*.
- **A2** "Region" means State.
- **A3** Queries are in English.
- **A4** There is a single local user.
- **A5** Internet access is available at setup and for the hosted LLM.
- **A6** A project-authored, disclosed runbook knowledge base citing public standards is an acceptable supplementary source.
- **A7** Severity relative to the dataset is acceptable because the data is synthetic.

2. Architecture



Offline flow (`python -m scripts.bootstrap`, no LLM):

1. Load and validate the CSV.
2. Normalize; compute percentile baselines; derive symptoms, categories and priority.
3. Train the ML models and write out-of-fold fault risk for every case.
4. Write one narrative per case and one chunk per runbook section.
5. Embed them (bge-small, CPU) into ChromaDB, and build a BM25 index over the narratives.

Online flow (`POST /troubleshoot`):

1. **Guardrails** screen the query before any LLM call.
2. **Gate.** If the LLM is available (key configured, circuit closed, daily spend cap not reached), the full multi-agent path runs; otherwise the reduced path runs.
3. **Classifier** (LLM) returns the category, symptoms, filters and scope. It can end the run by asking for clarification or rejecting the query as out of scope.
4. **Retrieve and gather evidence** (deterministic):
    - hybrid similar-case search;
    - KPI assessment against baselines;
    - segment statistics;
    - ML predictions;
    - impact and priority;
    - runbook sections chosen per agent role.
5. **Analyst** (LLM) ranks root-cause hypotheses, each with an evidence type and citations.
6. **Recommender** (LLM) writes ordered actions grounded in runbooks.
7. **Judge** (LLM plus deterministic rule checks) returns pass, revise (once) or fail. A final fail is shown as a low-confidence diagnosis with the judge's issues.
8. **Finalize:** strip unknown citations, and attach sources, usage and a trace.

Every LLM node has a deterministic fallback. Zero-key mode runs the same request with rules only.

Mandated spine. Retriever → LLM/RAG Pipeline → Agents → API Layer → User Interface, with the four agents and the judge loop inside Agents. The frontend talks to the backend only over HTTP (`/api/v1`).

Frontend (React, `frontend-react/`). Four areas: **Investigate** (describe an incident, watch the agents work live, read the report), **Cases** (similar-case search), **Simulate** (what-if KPIs, fault risk, throughput) and **Quality** (judge verdicts, feedback and the saved evaluation runs). The report is summary-first: diagnosis, priority and severity, impact, likely cause, escalation, the worst KPIs and the next three actions come first; causes, KPIs, similar cases, validation, sources and the agent trace sit in tabs. A presenter overview is in `docs/Architecture_Overview_React.md`.

3. Technology selection and justification

Concern	Choice	Why	Alternatives considered
Language	Python 3.12	Mandated language; best wheel coverage for torch, Chroma, scikit-learn	3.13 (riskier ML wheels)
Backend	FastAPI + Uvicorn	Pydantic schemas give validation and OpenAPI docs for free	Flask, Django REST Framework
Frontend	React 19 + TypeScript, Vite, TanStack Query; hand-written CSS tokens, hand-built charts	Full control over a dense, summary-first incident report with live agent progress; typed API contracts	Vue, Svelte, Next.js; a component kit (MUI)
LLM	**Claude Haiku 5.5 through the AICredits gateway** (OpenAI-compatible, prepaid credits); the direct Claude API and the NVIDIA API catalog remain config switches behind one client	Reliable JSON output and judging. Haiku 5.5 is ~10× cheaper per query than Sonnet 5.5 (§10); prepaid credits are a hard spending ceiling.	Sonnet 5.5 (evaluated, higher quality, $0.076/query); Opus 5.5; NVIDIA Nemotron (original runtime, slower, free tier)
Embeddings	`BAAI/bge-small-en-v1.5`	384-d, ~33M parameters, strong retrieval quality for its size, CPU-friendly, no key	`all-MiniLM-L6-v2` (faster, weaker); fastembed
Vector DB	ChromaDB (embedded, persistent)	No server, metadata `where` filters, cosine HNSW	Qdrant local (native hybrid), LanceDB, FAISS (no metadata filters)
Keyword search	bm25s	Millisecond BM25 over 100k documents, saved to disk, full score vectors so the metadata mask can be applied	rank_bm25 (≈0.3–0.5 s per query)
RAG framework	Thin custom pipeline	Filters must apply identically to dense and BM25 search; easier to explain and test	LangChain / LlamaIndex

Concern	Choice	Why	Alternatives considered
Agents	LangGraph	Typed shared state, conditional edges, a bounded critique-and-revise loop, Mermaid export	Plain orchestrator; CrewAI/AutoGen (chatty loops, weaker token control)
ML	scikit-learn HistGradientBoosting	Fast on 100k rows on CPU, native categoricals, no extra dependency	LightGBM / XGBoost (marginal gain)
Storage	SQLite	Runs, feedback, LLM response cache, spend ledger: zero setup	JSON files, Postgres
Evaluation	DeepEval 2.9.3 with a custom judge adapter; custom retrieval/classifier/guardrail/ML/judge evaluations	DeepEval is mandated	RAGAS (planned, cut for time)

4. Data understanding

Full details: `docs/Dataset_Details.pdf`. The design follows from these measured findings:

- **No free text at all**, so retrievable "documents" are generated from the KPIs (case narratives), and troubleshooting knowledge comes from nine authored runbooks.
- `Issue Reported` **is noise** (ROC-AUC 0.501), so it is never a label or a feature.
- **Five columns are formula-derived** (fault occurrence rate, throughput, critical alarms, authentication failures, call drops: R^2 0.94–1.00 from other columns). A naive model would be a tautology, so the ML uses leading indicators only (§7).
- **The data is synthetic.** Faker cities (38,066 unique), uniform distributions, no timestamp or cell ID. Trends cannot be verified and the answers say so; severity is percentile-relative (degraded > p75, critical > p90); the city filter is nearly useless, so state, season and weather matter.
- **KPIs are mostly independent of each other**, so root cause cannot be learned from correlations. Analysis is evidence-based hypothesis generation, and every hypothesis states its evidence type: data, knowledge, or both.

5. Retrieval strategy

- **Case narrative (template v1).** One document per record that names only its abnormal KPIs, each with severity word, value, percentile and synonyms (example in Dataset Details §4). Normal KPIs are left out, so BM25 gets meaningful terms and the embedding focuses on the symptom combination.
- **Chunking.**
  - *Record level for cases:* each record is short and self-contained.
  - *Section level for runbooks:* one chunk per H2 section, 81 chunks in all.
  - *Why:* agents retrieve runbook sections by role. The analyst gets the KPI signature and likely root causes; the recommender gets diagnostic checks, corrective actions and escalation criteria.
- **Metadata schema:** state, case-folded city, season, weather, primary and secondary categories, priority, fault-risk band, numeric KPIs. Filters apply identically to the dense path (an exact masked search over cached vectors when filters are selective) and to BM25.
- **Hybrid fusion.** Dense retrieval brings recall on paraphrases ("slow speeds" ≈ "reduced throughput"); BM25 brings precision on exact terms ("Snow", state codes, symptom words). Reciprocal-rank fusion (k = 60) combines them with no score normalization.
- **Rerank.** RRF score + 0.3 × symptom overlap with the query + 0.1 × feedback prior. Query expansion maps symptom synonyms to canonical terms.

- **Measured:** hybrid + rerank reaches Hit@5 1.0 and nDCG@10 0.953, against 0.913 for BM25 alone and 0.861 for dense alone.
- **No cross-encoder.** Web-QA rerankers aren't trained on templated KPI text and would add about 1–2 s of CPU latency; left as future work.

6. Agent design

What counts as an agent. A component is an agent only if it makes an LLM decision under ambiguity, has its own role and output contract, and its output changes the control flow. Guardrails, retrieval, KPI assessment, statistics, ML, impact and prioritization are exactly computable, so they are deterministic tools.

Agent	Output (JSON, schema-validated)	Reasoning	Fallback
Classifier	category, secondary categories, symptoms, filters, severity hint, scope, clarification question, confidence	off	rule-based classifier
Root-Cause & Impact Analyst	≤ 3 ranked hypotheses (likelihood, evidence type, supporting evidence, citations), contributing factors, impact narrative, data limitations	adaptive	runbook root causes + top anomalies + top-lift factors
Recommender	1–8 ordered steps (phase, rationale, expected KPI effect, verification, risk, change approval, citations), escalation	off	the runbook's corrective actions verbatim
Judge	4 scores (groundedness, KPI consistency, actionability, completeness), targeted issues, summary	adaptive	rule checks only, verdict "unvalidated"

- **State and blackboard.** LangGraph state carries the guardrail result, the classification, the evidence (cases, assessment, segment stats, predictions, impact, priority, runbook sections), the agent outputs, the judge's revision instructions per agent, warnings, usage and a node trace.
- **Judge loop.** Six deterministic rule checks run before the LLM judge: citations valid; every recommendation cites a runbook *(critical)*; hypotheses grounded; quoted numbers match the evidence; primary category addressed *(critical)*; 4–7 recommendations.
    - **Pass:** groundedness ≥ 0.70, all other scores ≥ 0.60, and no critical rule failure.
    - **Otherwise:** the issues go back to the targeted agent for **one** revision.
    - **A second fail** is returned as a low-confidence diagnosis, with the issues shown to the user.
- **Citations.** Every evidence item carries an ID (`CASE-…`, `KB-<category>-<section>`, or a data-section label such as `KPI ASSESSMENT` or `IMPACT & PRIORITY`). IDs that are not in the evidence are removed and reported.
- **Robustness.**
    - *JSON handling:* JSON output with one repair attempt; tolerant normalization of known shape slips, logged with the raw reply.
    - *Cache:* only validated replies are cached.
    - *Circuit breaker:* authentication errors open it until restart; 3 consecutive failures open it for 5 minutes.
    - *Per-run budget:* 12 LLM calls and 50,000 tokens.
    - *Daily spend cap:* default $1.00.
- **Token budget per call (output):** classifier 400; analyst 900 + 4,096 reasoning; recommender 2,500; judge 500 + 4,096 reasoning.
- **Difference from the plan.** The analyst does not choose its own tool calls. The graph invokes every deterministic tool before the analyst runs, so the analyst's "tool-calling round" was deferred. The classifier's correction memory (few-shot examples from corrected queries) was also cut.

7. ML design, prioritization and impact

- **Targets.**
  - *Fault risk:* P(fault occurrence rate ≥ the training p75 of 48.15%).
  - *Performance:* throughput (Mbps).
- **Feature rule.** A column is excluded from features if it reaches $R^2 \geq 0.15$ when predicted from the independently generated ("root") columns, is the target, is label noise or is an identifier. Temperature and humidity are kept as context: the generator derives them from season by design. Evidence: `docs/assets/eda/leakage_audit.csv`.
- **Results.**
  - *Fault risk:* ROC-AUC 0.874, Brier 0.120 (out-of-fold AUC 0.875).
  - *Throughput:* R^2 0.328, MAE 161.8 Mbps.
  - *Risk bands* (Low < 0.30 ≤ Medium < 0.60 ≤ High): actual high-fault rates 7.9% / 44.8% / 73.1%.
- **Prioritization.**
  - *Score:* 0.4 × severity + 0.3 × impact + 0.3 × risk. Severity is a weighted band sum (service-affecting KPIs weigh 1.5×); impact is percentile-based.
  - *Levels:* corpus-quantile cut-offs (P1 top 5%, P2 next 20%, P3 next 40%, P4 the rest).
  - *Overrides* to at least P2: critical availability; critical alarms with fault rate; security incidents with authentication failures.
- **Impact.** Severe when users ≥ p90 and a service-affecting KPI is critical; High when either holds; Moderate when a service-affecting KPI is degraded; Low otherwise.

8. Guardrails and safety

Order: normalization → gibberish → prompt injection → PII redaction → domain relevance.

- **Why first:** guardrails run before any LLM, so rejected input never reaches a prompt.
- **Injection:** pattern rules; rejected with `PROMPT_INJECTION`.
- **PII:** email addresses, phone numbers and long ID numbers (9+ digits) are redacted, not rejected, with a warning.
- **Domain relevance:** embedding similarity to the domain. In scope ≥ 0.65; out of scope ≤ 0.55 (rejected with `OUT_OF_SCOPE`). The uncertain band in between is decided by the classifier.
- **Measured:** 12/12 should-block cases blocked, 0/27 false positives.
- **Answer-side safety:**
  - citations are validated;
  - disruptive actions are flagged *requires change approval*;
  - relative severity, imputed KPIs and the lack of time series are stated as warnings or data limitations.

9. Feedback loop

- **Capture** (`POST /feedback`): an overall rating, "helpful", a corrected category, a comment, and per-case relevance votes, linked to the run.
- **Use online.** A Beta(1,1)-smoothed prior per (case, category) enters the rerank with weight 0.1, so feedback nudges rankings but a single vote cannot dominate.
- **Use offline.** Runs rated ≤ 2 are exported to `tests/test_cases/feedback_regressions.json` for re-testing; `GET /feedback/summary` shows rating trends and the most down-voted cases.
- **Measured** (`run_feedback_sim.py`): voted queries improve by +0.006 nDCG@10 (4 improved, 0 regressed); control queries are unchanged.

10. Evaluation methodology and headline results

Full report: `docs/Evaluation_Report.pdf`. The suite covers search, classification, guardrails, ML, prioritization, the feedback loop, answer quality against a basic-RAG baseline (DeepEval) and the judge agent's ability to catch planted errors. The LLM-based parts were run on two configurations:

	Best capability (2026-10-09)	**Demo setup (2026-10-11)**
Agents / evaluation judge	Claude Sonnet 5.5 / Claude Opus 5.5	Claude Haiku 5.5 via AICredits / gpt-oss-120b
Targets met	26 of 28	27 of 28
Classifier accuracy	0.955	0.955
Median time per analysis	31 s	35 s
Judge verdicts (11 queries)	11 pass	9 pass, 2 low confidence (3 revised)
Actionability, basic RAG → multi-agent	0.60 → 0.84	0.66 → 0.99
Planted errors caught by the judge agent	12/12, no false alarms	12/12, no false alarms

Search (Hit@5 1.0, nDCG@10 0.953), guardrails (12/12 blocked, 0 false positives), ML (fault-risk ROC-AUC 0.874) and prioritization (6/6) do not use a model and are identical in both runs. Answer-quality scores come from different judges and are compared within a run. The missed targets are context metrics that reflect the evaluation's context layout and a runbook section the system never retrieves, not answer quality.

Cost decision. The Sonnet/Opus evaluation cost about $11.4; the demo-setup evaluation about $0.27. The app runs Haiku 5.5 (≈ $0.007 per analysis against $0.076): same classification accuracy, less consistent answers, which the judge revises or labels low confidence.

11. Challenges, trade-offs, limitations and future work

Challenges and trade-offs

- *Synthetic data* forced percentile-relative severity, generated narratives and an authored knowledge base. Honest ML scores (AUC 0.874) were preferred over leaky "perfect" ones.
- *Agents vs one call:* 4 LLM calls instead of 1, bought for measurably better actionability and an independent check of every answer.
- *Provider migration (NVIDIA → Claude)* cut latency from a 76 s to a 31 s median and removed JSON failures. Model choice then became a cost trade-off, settled on Haiku 5.5 through a prepaid gateway.
- *Cost control:* the response cache, a per-run budget, a daily spend cap, and a reduced mode that always answers.

Limitations

- Synthetic data with no time axis or topology.
- Single labeller and small evaluation sets.
- The runtime judge shares the agents' model.
- Runbook verification sections are not retrieved, so verification steps are model-written.
- No authentication.

Future work

- Real operator data with timestamps and cell IDs, enabling time-series trends and forecasting.

- Retrieve runbook verification sections for the recommender.
- A cross-encoder rerank ablation.
- Analyst-chosen tool calls.
- Authentication and multi-user support.
- A production vector database.